

CS3002E Compiler Design

Monsoon 2025

Lecture #16

Bottom-Up Parsing - 5: SLR Parsing

Department of Computer Science & Engineering
NIT Calicut

Example: a grammar that is not LR(0)

$$S' \rightarrow S$$

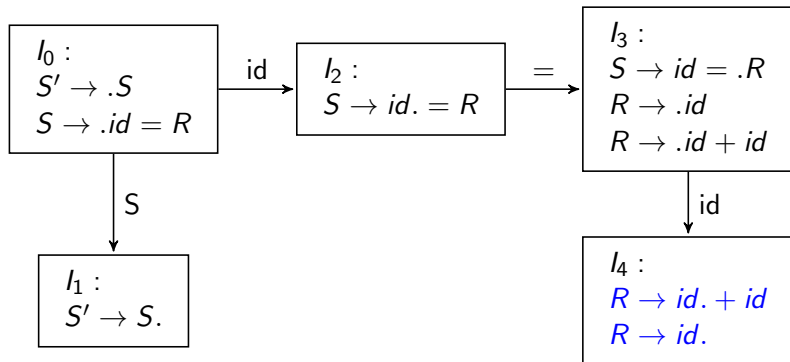
$$S \rightarrow id = R$$

$$R \rightarrow id + id$$

$$R \rightarrow id$$

LR(0) Automaton (not all states included)

0 : $S' \rightarrow S$ 1 : $S \rightarrow id = R$ 2 : $R \rightarrow id + id$ 3 : $R \rightarrow id$



shift/reduce conflict

0 : $S' \rightarrow S$ 1 : $S \rightarrow id = R$ 2 : $R \rightarrow id + id$ 3 : $R \rightarrow id$

$I_4 :$

$R \rightarrow id. + id$

$R \rightarrow id.$

- the item $R \rightarrow id.$ indicates a **reduction** when the parser is in state I_4 , whatever be the look-ahead symbol
- but the item $R \rightarrow id. + id$ in the same state indicates a **shift** on input symbol $+$

Two possible actions in the same state - a **shift/reduce** conflict.

Example: a grammar that is not LR(0)

0 : $S' \rightarrow S$ 1 : $S \rightarrow id = R$ 2 : $R \rightarrow id + id$ 3 : $R \rightarrow id$

State	id	=	+	\$	R	S
0	s2					1
1				acc		
			...			
4	r3	r3	r3/s5	r3		
			...			

- *shift/reduce* conflict in state 4
- Grammar is not LR(0)

A method to resolve the conflict

$$0: S' \rightarrow S \quad 1: S \rightarrow id = R \quad 2: R \rightarrow id + id \quad 3: R \rightarrow id$$

I_4 :

$R \rightarrow id. + id$

$R \rightarrow id.$

The parser reaches state I_4 after reading the prefix $id = id$ from the input. In this state,

- reduction $R \rightarrow id$ is valid only if lookahead is \$

$$S' \$ \Rightarrow S \$ \Rightarrow id = R \$ \Rightarrow id = \underline{id} \$$$

- if lookahead is +, the parser should shift to obtain the prefix $id +$ of handle $id + id$

$$S' \$ \Rightarrow S \$ \Rightarrow id = R \$ \Rightarrow id = \underline{id + id} \$$$

SLR Parsing

- SLR (Simple LR) - more powerful than LR(0).
- Reduction $A \rightarrow \alpha$ only when lookahead is in FOLLOW(A).

SLR Parsing Table

0 : $S' \rightarrow S$ 1 : $S \rightarrow id = R$ 2 : $R \rightarrow id + id$ 3 : $R \rightarrow id$

State	id	=	+	\$	R	S
0	s2					1
1				acc		
			...			
4			s5	r3		
			...			

- In state 4, reduction by $R \rightarrow id$ only if lookahead is \$
\$ $\in FOLLOW(R)$
- No conflicting entries in SLR Parsing table
- Grammar is SLR(1) (or just SLR)¹

¹when value of k is not mentioned, it should be taken as 1

SLR Parsing

Show the SLR Parsing table entries for:

$0: S' \rightarrow S$ $1: S \rightarrow L = R$ $2: L \rightarrow id$ $3: R \rightarrow num$

Compute $FOLLOW(S)$, $FOLLOW(L)$ and $FOLLOW(R)$.

SLR Parsing Table

0 : $S' \rightarrow S$ 1 : $S \rightarrow L = R$ 2 : $L \rightarrow id$ 3 : $R \rightarrow num$

state	id	=	num	\$	L	R	S
0	s3				2		1
1				acc			
2		s4					
3		r2					
4			s6			5	
5				r1			
6				r3			

Check if the given grammar is LR(0)

$$E' \rightarrow E$$

$$E \rightarrow T + E \mid T$$

$$T \rightarrow id$$

A grammar that is not LR(0)

$$E' \rightarrow E$$

$$E \rightarrow T + E \mid T$$

$$T \rightarrow id$$

LR(0) automaton has a state with the following set of items:

$$E \rightarrow T. + E$$

$$E \rightarrow T.$$

In this state, the parser has conflicting actions - either *shift* $+$ or *reduce* using production $E \rightarrow T$ - a *shift/reduce* conflict

SLR Parsing

$$E' \rightarrow E$$

$$E \rightarrow T + E \mid T$$

$$T \rightarrow id$$

$$E \rightarrow T. + E$$

$$E \rightarrow T.$$

SLR parser reduces only when the lookahead is in $FOLLOW(E)$.

SLR Parser

$$0: E' \rightarrow E$$

$$1: E \rightarrow T + E$$

$$2: E \rightarrow T$$

$$3: T \rightarrow id$$

$$\begin{array}{l} E \rightarrow T. + E \\ E \rightarrow T. \end{array}$$

- reduction by production $E \rightarrow T$ is valid only when lookahead is \$ (\$ is the only symbol in $FOLLOW(E)$)
- SLR parser reduces by $E \rightarrow T$ only if lookahead is \$
- parser shifts if the lookahead is +

SLR Parsing Table

Let $C = I_0, I_1, \dots, I_n$, be the canonical collection of sets of LR(0) items. For each set I_i , the parser has a corresponding state i . The parsing table has two parts - ACTION and GOTO. The entries for state i are as follows:

- If $[A \rightarrow \alpha.a\beta]$ is in I_i , and $Goto(I_i, a) = I_j$, set $ACTION[i, a]$ to s_j (shift and goto state j)
- If $[A \rightarrow \alpha.]$ is in I_i , and $A \neq S'$, set $ACTION[i, a]$ to r_j (reduce using production numbered j , assuming $A \rightarrow \alpha$ is numbered j) for every symbol $a \in FOLLOW(A)$
- If $[S' \rightarrow S.]$ is in I_i , set $ACTION[i, \$]$ to *accept*
- If $Goto(I_i, A) = I_j$ then set $GOTO(i, A)$ to j
- All undefined entries are marked *error* (blank entries)

Exercise

Check if each of the following grammars is (a) LR(0) (b) SLR?

1. $E \rightarrow E + T \mid T$
 $E \rightarrow T$
 $T \rightarrow id$

2. $S \rightarrow L = R \mid R$
 $L \rightarrow *R \mid id$
 $R \rightarrow L$

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.
- Appel A. W., Modern Compiler Implementation in Java, Cambridge University Press, 2000.

Further reading:

- ALSU chapter 4, section 4.6.4, Appel A. W, section 3.3